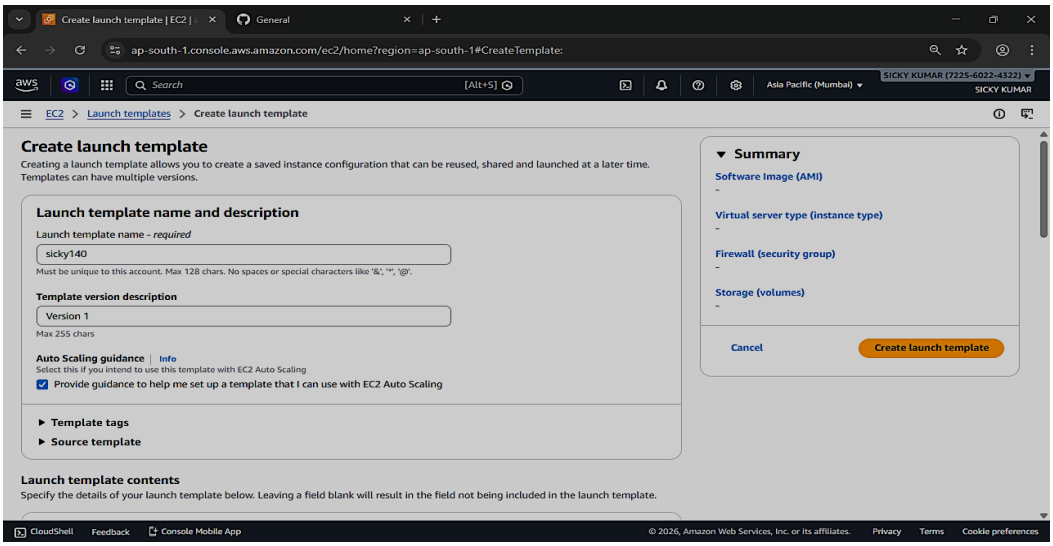
Assignment No: 11

Problem Statement: Build scaling plans in AWS that balance the load on different EC2 instances.

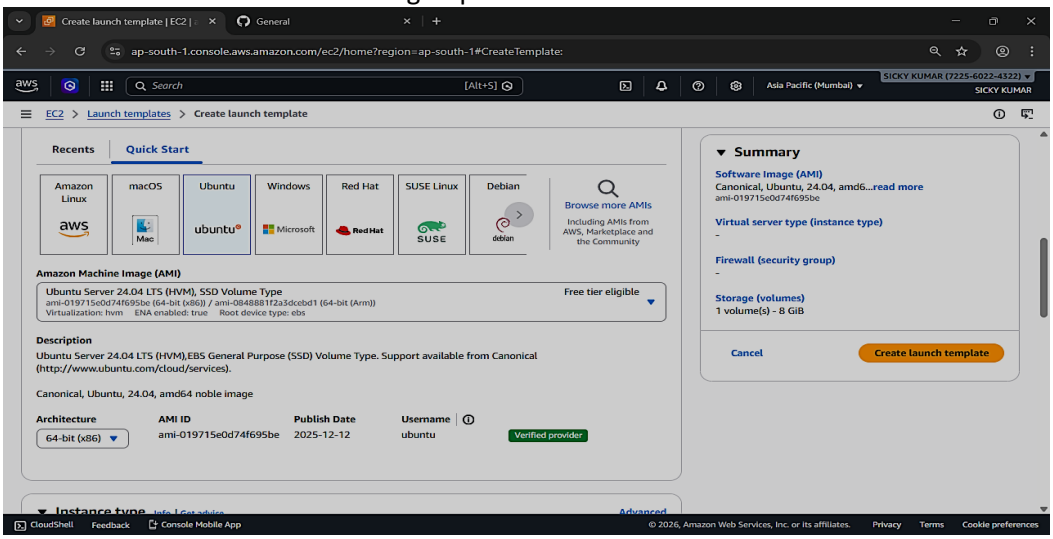
Step 1: Create Launch Template

A new Launch Template named sicky140 is created. The launch template stores configuration details such as AMI, instance type, key pair, security group, and user data. It acts as a blueprint for launching EC2 instances automatically. This ensures that every instance created later has identical configuration.



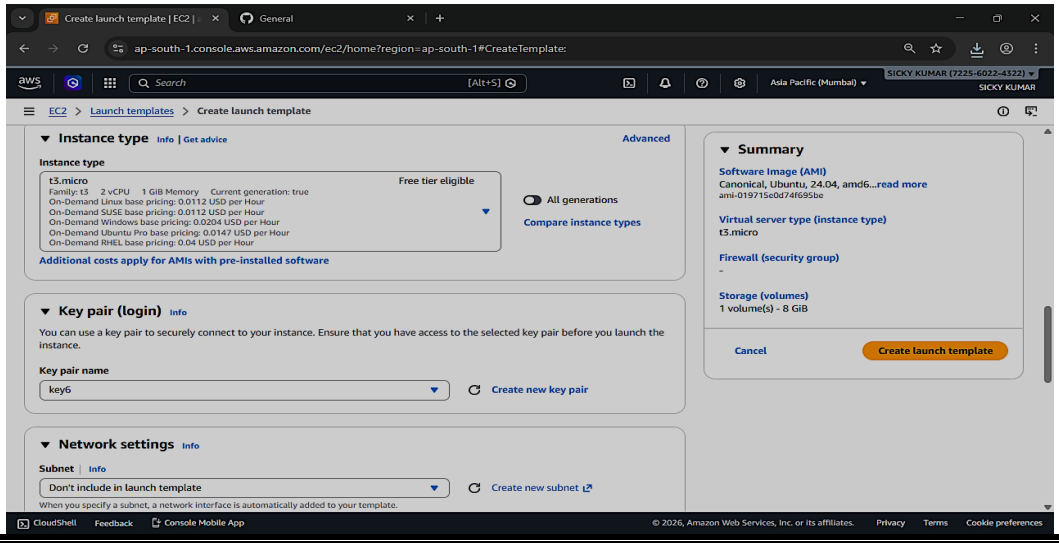
Step 2: Select Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS is selected as the Amazon Machine Image. The AMI provides the operating system environment for EC2 instances. Selecting a specific AMI ensures all instances run the same OS and environment.



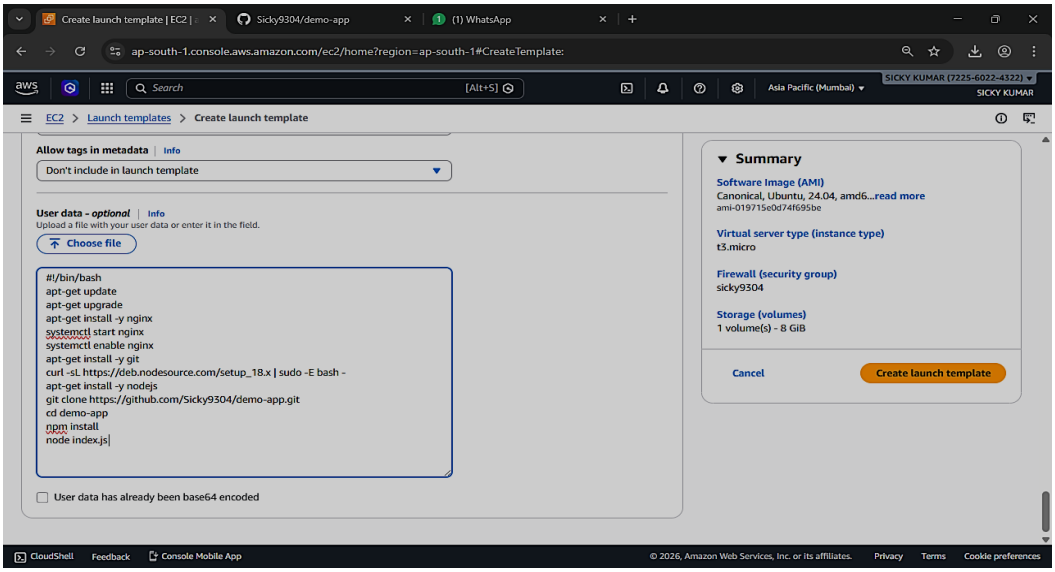
Step 3: Choose Instance Type and Key Pair

The instance type **t3.micro** is selected for cost-effective performance. This instance type provides suitable CPU and memory for lightweight applications. A key pair named **key6** is selected to enable secure SSH access to the instances. This ensures secure remote management of the server.



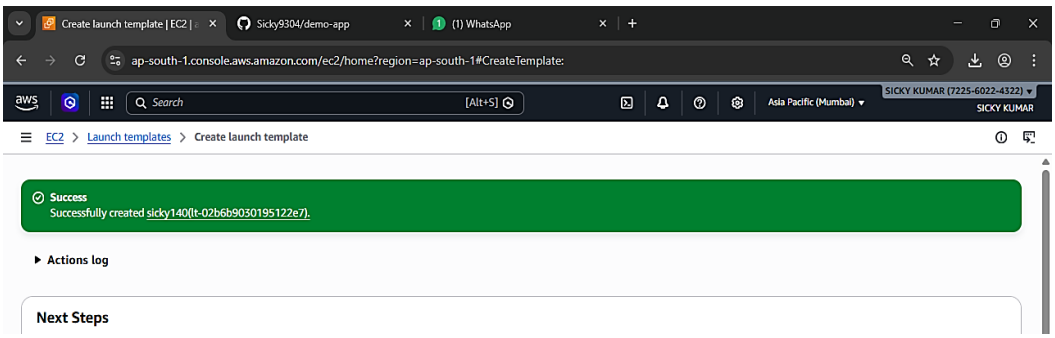
Step 4: Add User Data Script

A user data script is added in the launch template configuration. This script runs automatically when an EC2 instance starts for the first time. It installs required software and performs server setup automatically. This reduces manual configuration and ensures automation.



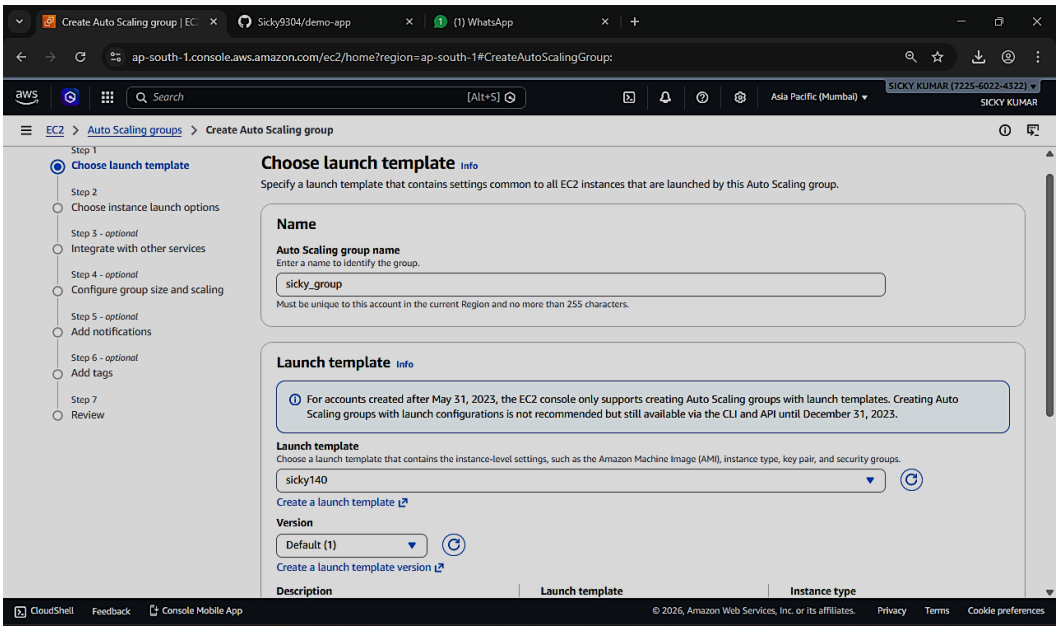
Step 5: Launch Template Creation Completed

The launch template is successfully created and saved. It is now available for use in the Auto Scaling Group configuration. This template will control how future EC2 instances are launched.



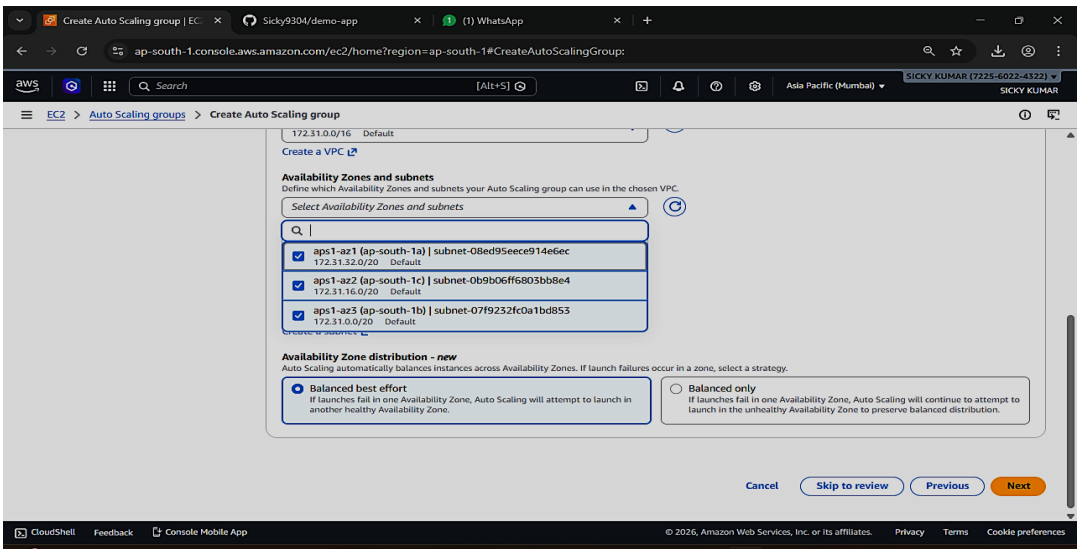
Step 6: Create Auto Scaling Group

An Auto Scaling Group named **sicky_group** is created. The previously created launch template is selected for this group. The Auto Scaling Group manages multiple EC2 instances automatically. It ensures that the required number of instances are always running.



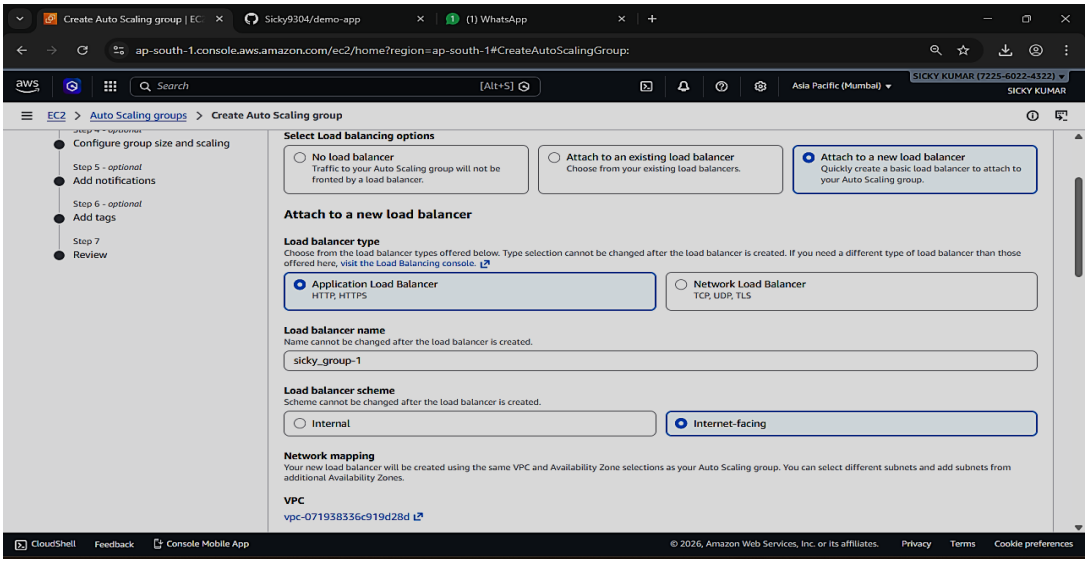
Step 7: Select Availability Zones and Subnets

Multiple Availability Zones and subnets are selected. This distributes EC2 instances across different physical data centers. If one zone becomes unavailable, instances in other zones continue running. This improves high availability and fault tolerance.



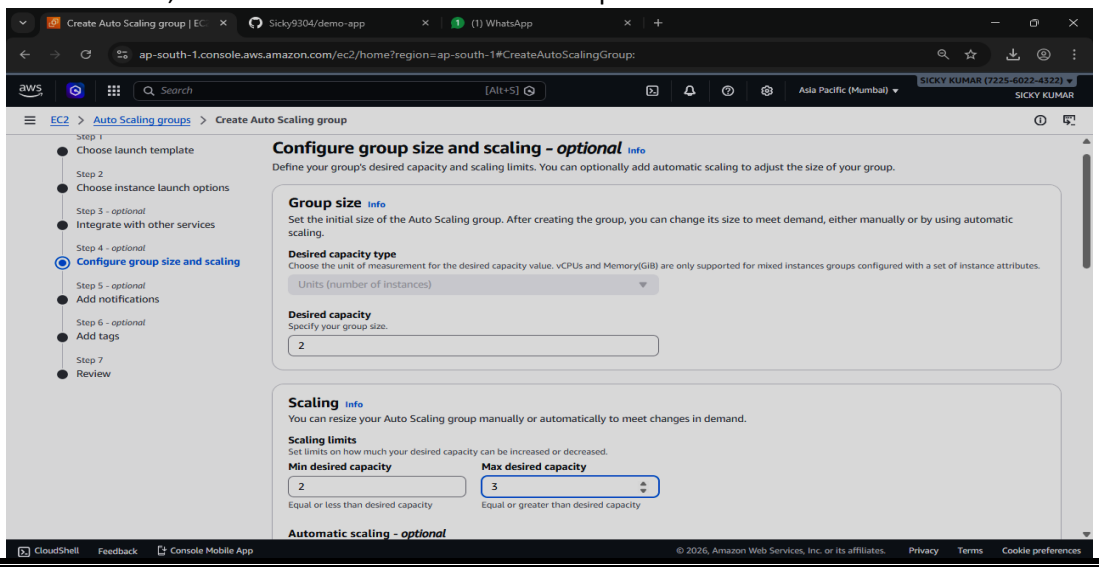
Step 8: Attach Application Load Balancer

A new **Application Load Balancer** is attached to the Auto Scaling Group. The scheme is configured as **Internet-facing** to allow public access. The load balancer distributes incoming traffic evenly among all healthy EC2 instances. This prevents any single instance from becoming overloaded.



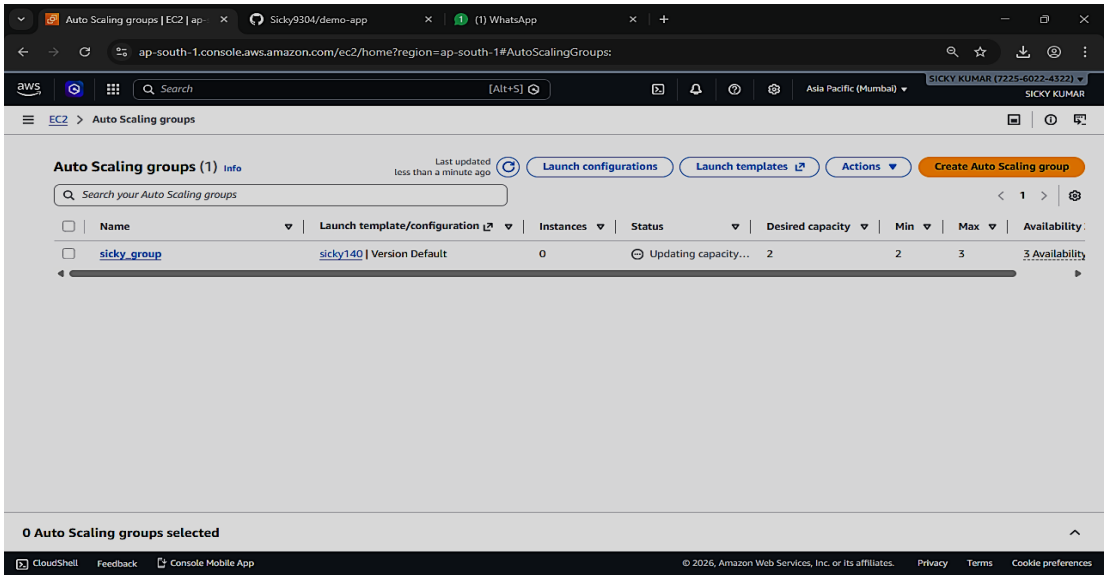
Step 9: Configure Group Size and Scaling Policy

The desired capacity is set to 2 instances, with a minimum of 2 and a maximum of 3. A target tracking policy is configured using Average CPU utilization (20%). If CPU usage increases, a new instance is launched automatically, and if it decreases, extra instances are terminated to optimize resources.



Step 10: Auto Scaling Group Created

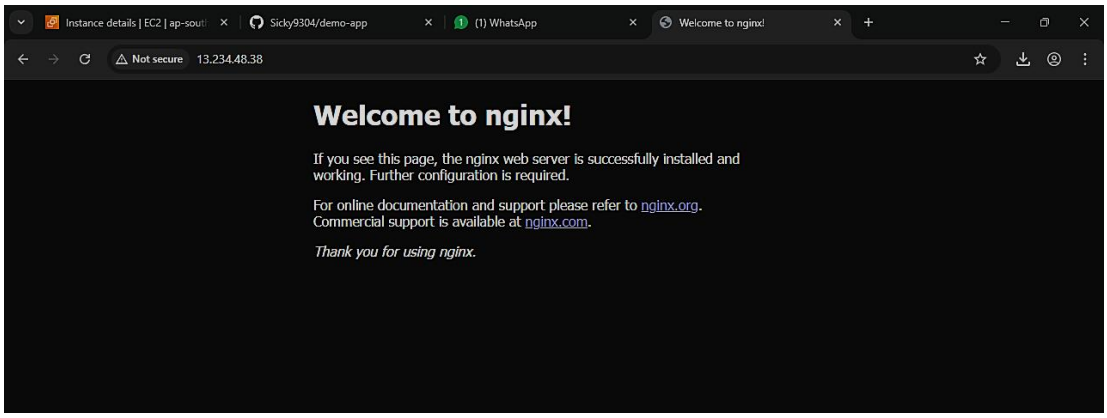
This image shows the Auto Scaling Group list page. It confirms that **sicky_group** is created successfully. Desired capacity shows 2 instances. This means scaling configuration is active.



Output Verification

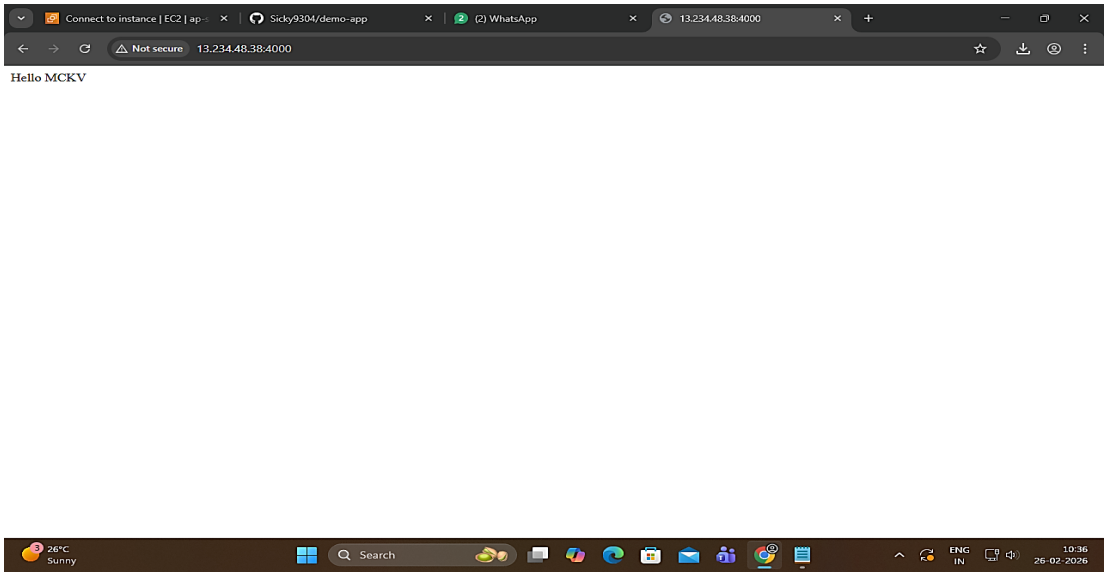
Step 11: Nginx Welcome Page

This image shows the default Nginx page. It confirms that the web server is installed successfully. The EC2 instance is running properly.



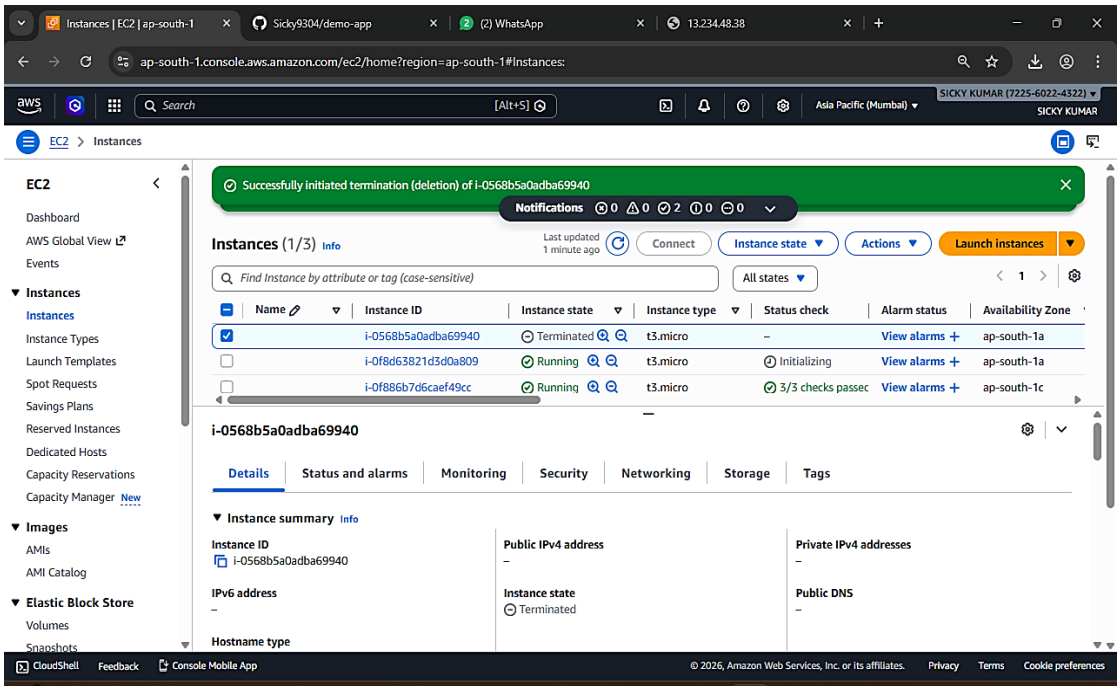
Step 12: Application Running on Port 4000

This image shows “Hello MCKV” on port 4000. It confirms the Node.js application is running successfully. The deployment was successful.



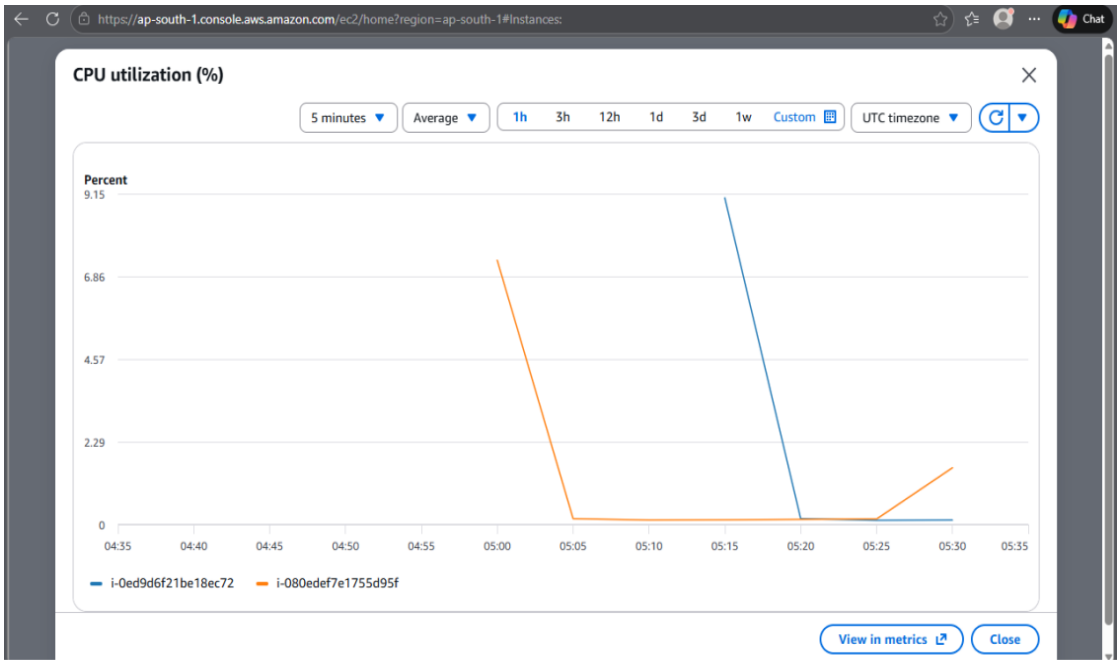
Step 14: Check Working of Load Balancing

Even after one instance was terminated, the Auto Scaling Group automatically launched new instances to maintain the required number of running servers, ensuring continuous availability and load balancing.



Step 14: Check Server Load By Graph

If we need to check for server's load, we can check from the Monitoring section from below,



[We have successfully Completed this Assignment and also manages the load balancing.]